

Login API - Hash y Session Storage

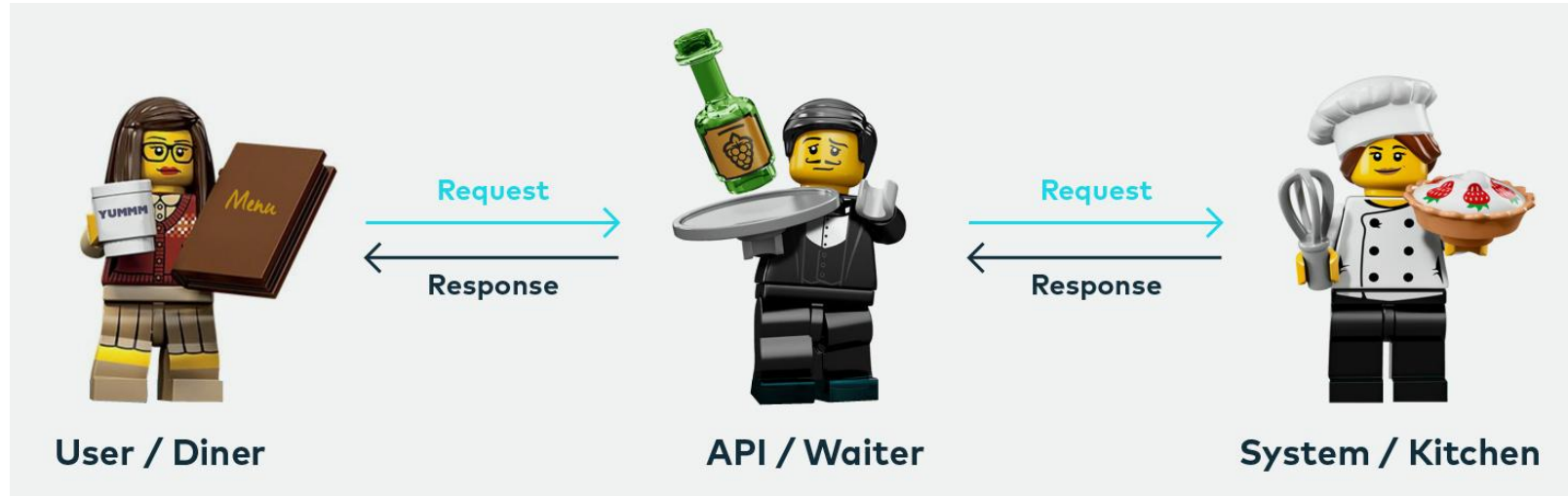
Diseño de Software

TICS316-2-2025

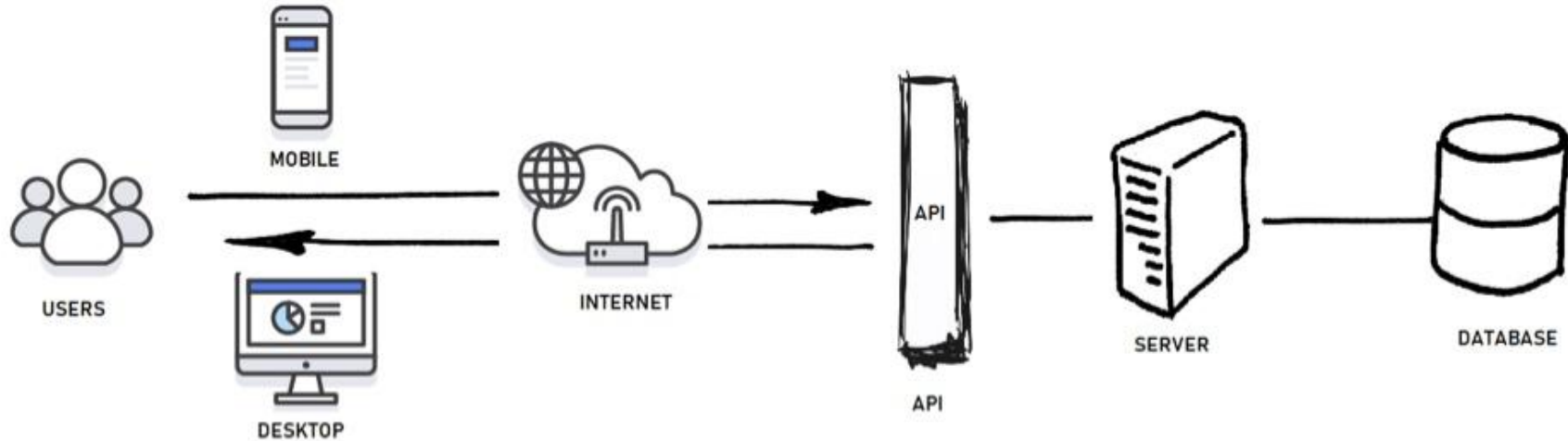
Agenda

- 1 API**
Arquitectura, Usos y ejemplos
- 2 Login + Hash**
Cifrado de contraseña para almacenamiento seguro
- 3 JWT**
JSON Web Token
- 4 Google OAuth**
Sistemas de autenticación alternativo
- 5 Ejemplo de código - NEXT.js**

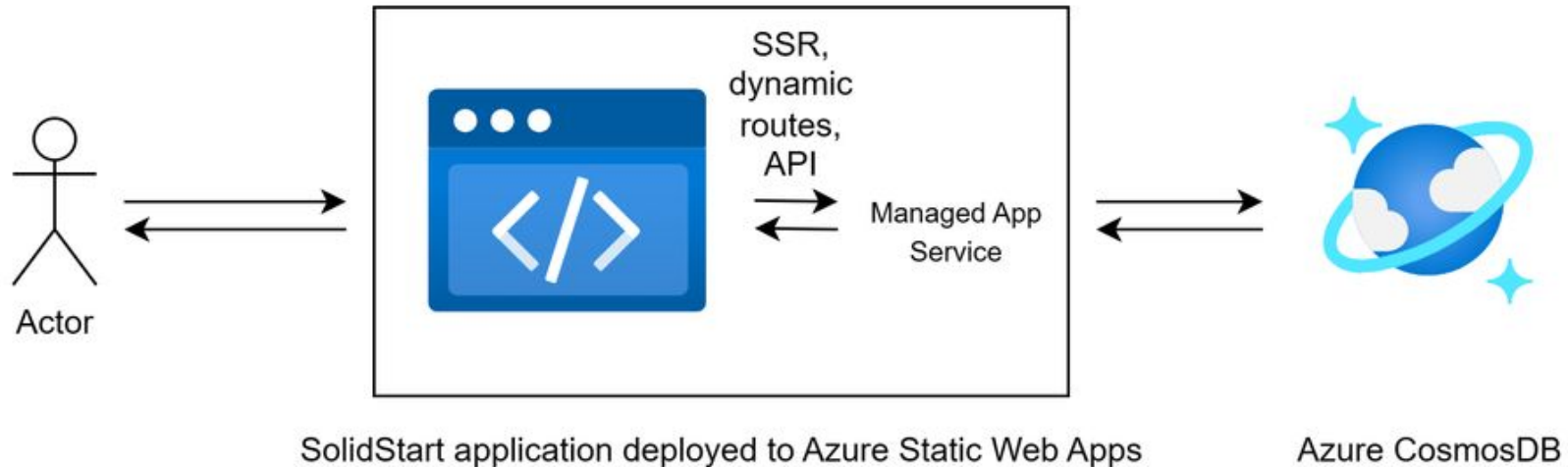
¿Dónde se ubican las APIs?



¿Dónde se ubican las APIs?



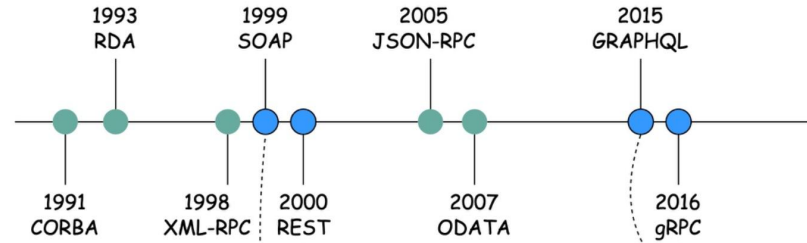
NEXT.js Next.js application



SolidStart application deployed to Azure Static Web Apps

Azure CosmosDB

El protocolo de API más común es “**API REST**”, la mayoría de aplicaciones y páginas web las utilizan. Estaremos profundizando su uso durante el curso. Por otro lado, buscaremos entregar más ejemplos y casos de uso sobre **GraphQL**, debido a sus aplicaciones en microservicios.

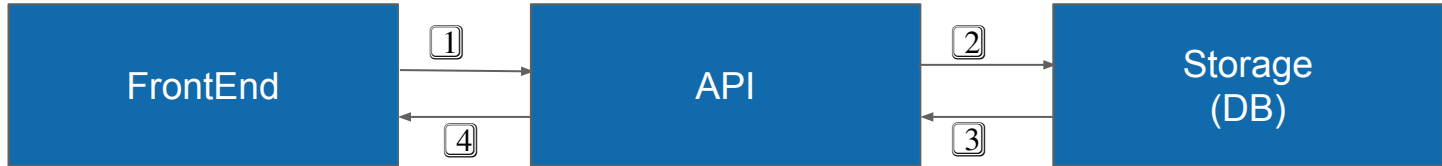


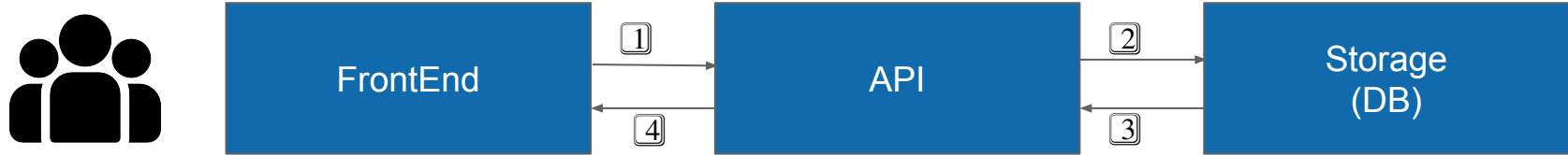
	SOAP (Simple Object Access Protocol)	REST (REpresentational State Transfer)	GraphQL	RPC (Remote Procedure Call)
Organized in terms of	enveloped message structure	compliance with six architectural constraints	schema & type system	local procedure call
Format	XML only	XML, JSON, HTML, plain text	JSON	JSON, XML, Protobuf, Thrift, FlatBuffers
Learning curve	Difficult	Easy	Medium	Easy
Community	Small	Large	Growing	Large
Use cases	- payment gateways - identity management - CRM solutions - financial and telecommunication services - legacy system support	- public APIs - simple resource-driven apps	- mobile APIs - complex systems - micro-services	- command and action-oriented APIs - high performance communication in massive micro-services systems

Agenda

- 1 API
Arquitectura, Usos y ejemplos
- 2 **Login + Hash**
Cifrado de contraseña para almacenamiento seguro
- 3 JWT
JSON Web Token
- 4 Google OAuth
Sistemas de autenticación alternativo
- 5 Ejemplo de código - NEXT.js

¿Cómo funciona un login?





Ejemplo:

FrontEnd - React, página web. API con Node.js, una URL. Storage, una base de datos SQLite.

0.- Cliente/Usuario ingresa su nombre de usuario y contraseña.

1.- El frontend prepara la información para utilizarla en una API específica, en este caso /login

2.- La API una vez recibida la información, debe confirmar que el nombre de usuario y contraseña coinciden, por lo que realiza la primera consulta a la base de datos. ¿Cuál es la contraseña del usuario?

3.- La base de datos entrega lo solicitado, la API compara la contraseña.

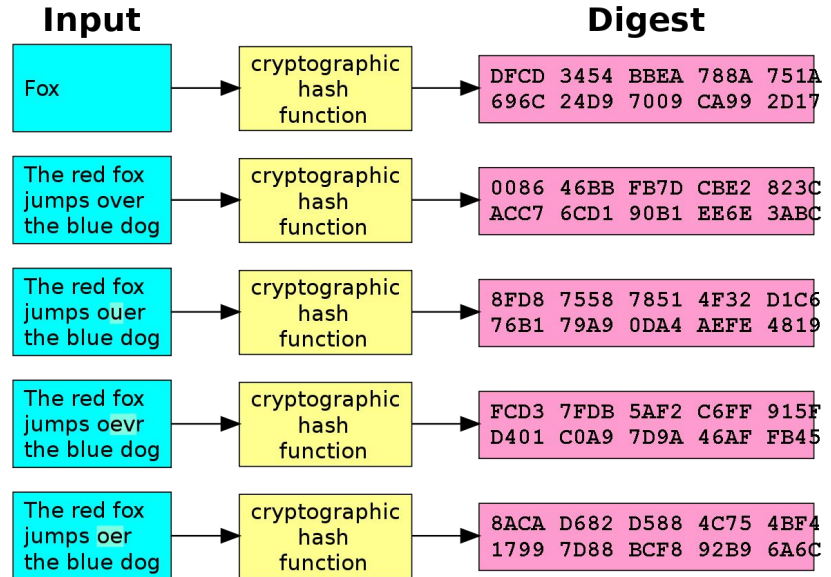
4.- Se entrega la información al usuario por medio de la respuesta de la API, puede decir que salió todo ok, la clave era correcta y entregarle por ejemplo el nombre y apellido del usuario o también podrían ocurrir uno o más errores que finalmente se le deben mostrar al usuario.



Una función **HASH** o “hashing” se refiere a un algoritmo **determinístico, unidireccional**, esta característica nos ayuda, ya que, es prácticamente imposible saber cuál es la palabra o frase inicial. Además, como podemos apreciar en la imagen del costado, una ligera variación del texto de entrada crea un resultado completamente distinto.

Por supuestos existen distintas funciones y con el pasar de los años, las más antiguas ya quedaron obsoletas. ¿Por qué?

¿Cómo nos afecta la computación cuántica?



Sobre la seguridad

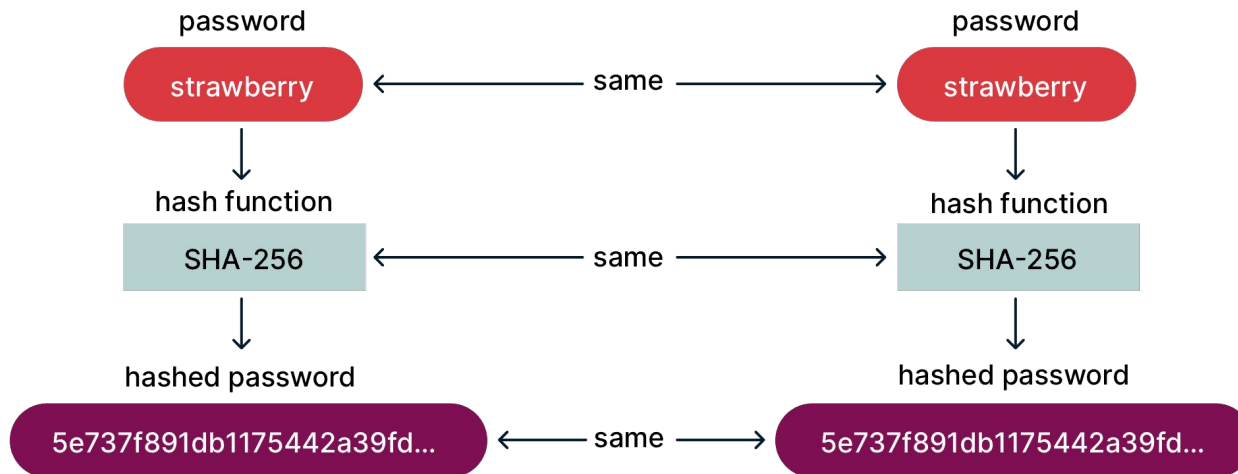
En general está limitada al largo de la llave de cifrado, por ejemplo si la llave es de 256 bits, la fuerza del cifrado no puede ser mayor a 256 bits.

Aquí es donde nos centramos en hacer que las combinaciones sean tantas, que tengan que tardar billones de años para descifrar el *plaintext*.

Today's encryption algorithms can be broken. Their security derives from the wildly impractical lengths of time it can take to do so. Let's say you're using a 128-bit AES cipher. The number of possible keys with 128 bits is 2 raised to the power of 128, or 3.4×10^{38} , or 340 undecillion. Assuming no information on the nature of the key is available (such as the fact that the owner likes to use his or her children's birthdays), a code-breaking attempt would require testing each possible key until one was found that worked. Assuming that enough computing power was amassed to test 1 trillion keys per second, testing all possible keys would take **10.79 quintillion years**. This is about 785 million times the age of the visible universe (13.75 billion years). On the other hand, you might get lucky in the first 10 minutes. But using **quantum technology** with the same throughput, exhausting the possibilities of a **128-bit AES key would take about six months**. If a quantum system had to crack a **256-bit key**, it would take about **as much time as a conventional computer needs to crack a 128-bit key**. A quantum computer could crack a cipher that uses the RSA or EC algorithms almost immediately.

— Lamont Wood

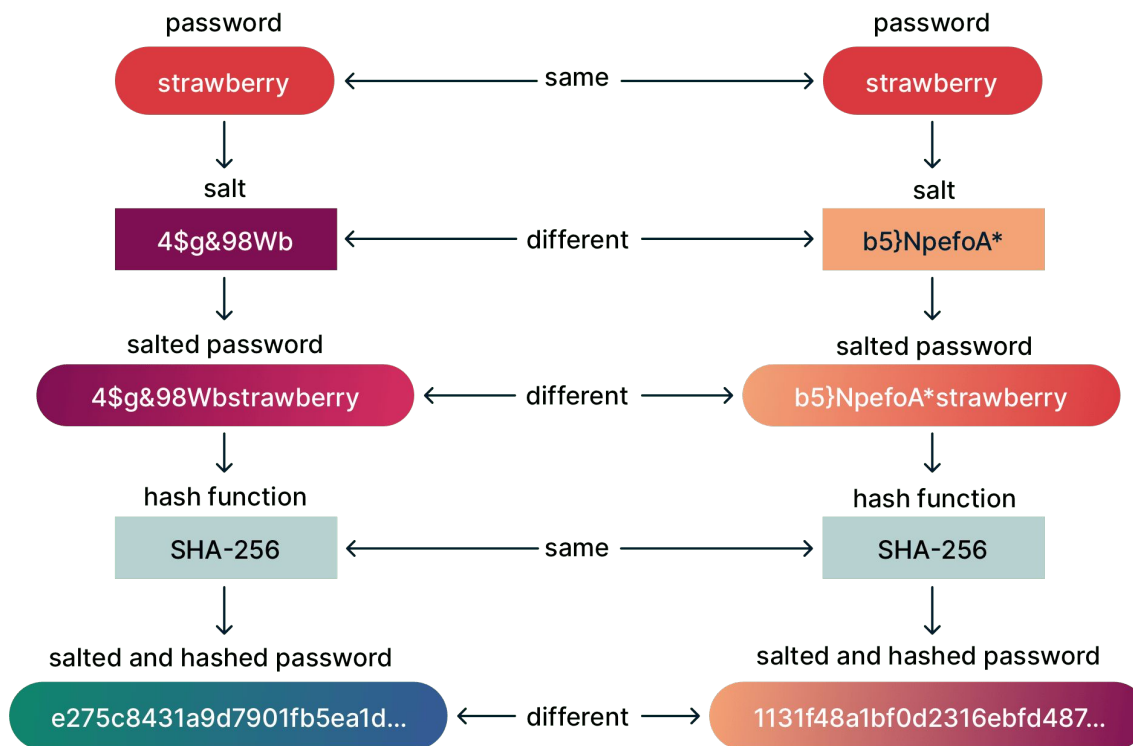
¡Tenemos un problema! Al ser un algoritmo determinístico, quiere decir que para el mismo *input*, nos entregará el mismo texto cifrado. Entonces dos usuarios que colocaron la contraseña 123456, tendrán en la base de datos el mismo texto cifrado.



Generamos esta sal, que no es nada más que un string de caracteres aleatorios. Decidimos si agregarlo al inicio o al final de la contraseña y lo más importante, lo guardamos en la base de datos. Ahora ya no comparamos solo el hash, sino que el “salted hash”. Muchas veces a esto se le agrega que puedes utilizar varias funciones de hash al mismo tiempo.

SHA(SHA(SHA(clave+sal)))

Esto hará nuestro login bastante más robusto.



Agenda

- 1 API
Arquitectura, Usos y ejemplos
- 2 Login + Hash
Cifrado de contraseña para almacenamiento seguro
- 3 JWT
JSON Web Token
- 4 Google OAuth
Sistemas de autenticación alternativo
- 5 Ejemplo de código - NEXT.js

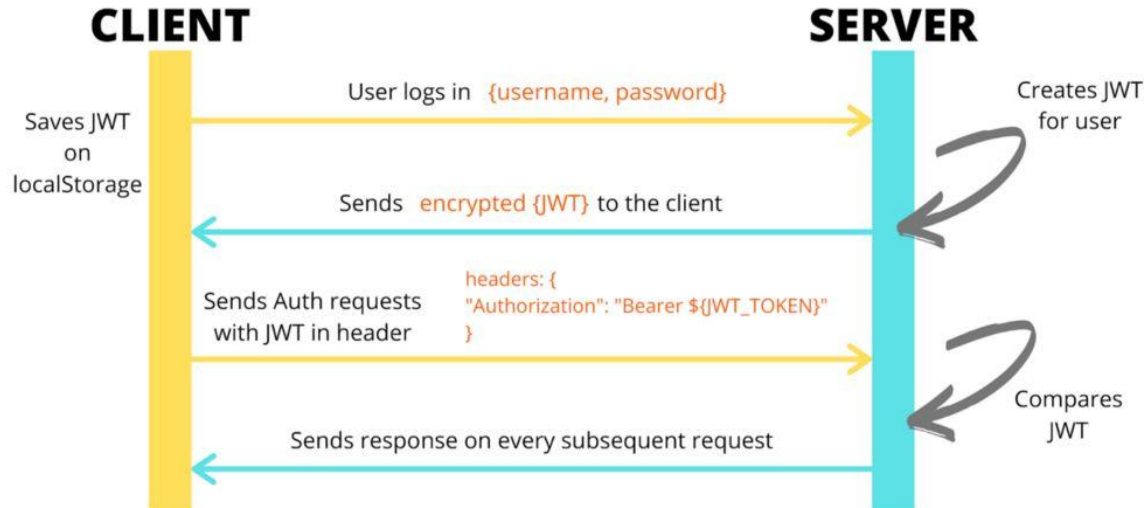
¿Quién almacena la sesión?
El Cliente (Frontend) o El Servidor
Stateless Stateful

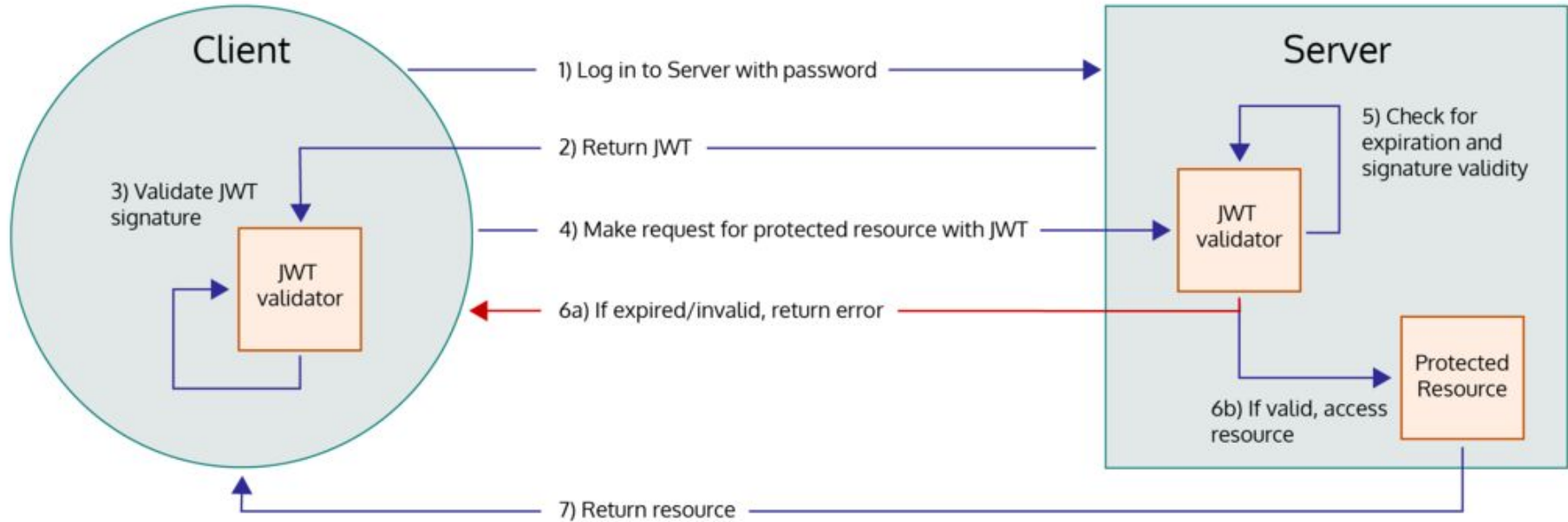
Ambas son opciones válidas, dependerá del caso de uso cuál vamos a preferir.

Siempre debemos tomar en cuenta que la **base de datos** mantendrá una versión **HASH** de nuestra contraseña, nunca en texto plano. Además de cumplir con otros principios básicos como el de **mínimo privilegio**.

Tokens	JWT
Stored server-side	Stored client-side
Easily revoked	Difficult to revoke
Require server-side storage	Stateless
Suitable for real-time	Better for mobile or SPA
Updates	Multiple domains or microservices
Scalable with Redis or Memcached	Requires distributed signing or validation
More secure	Faster and easier to implement

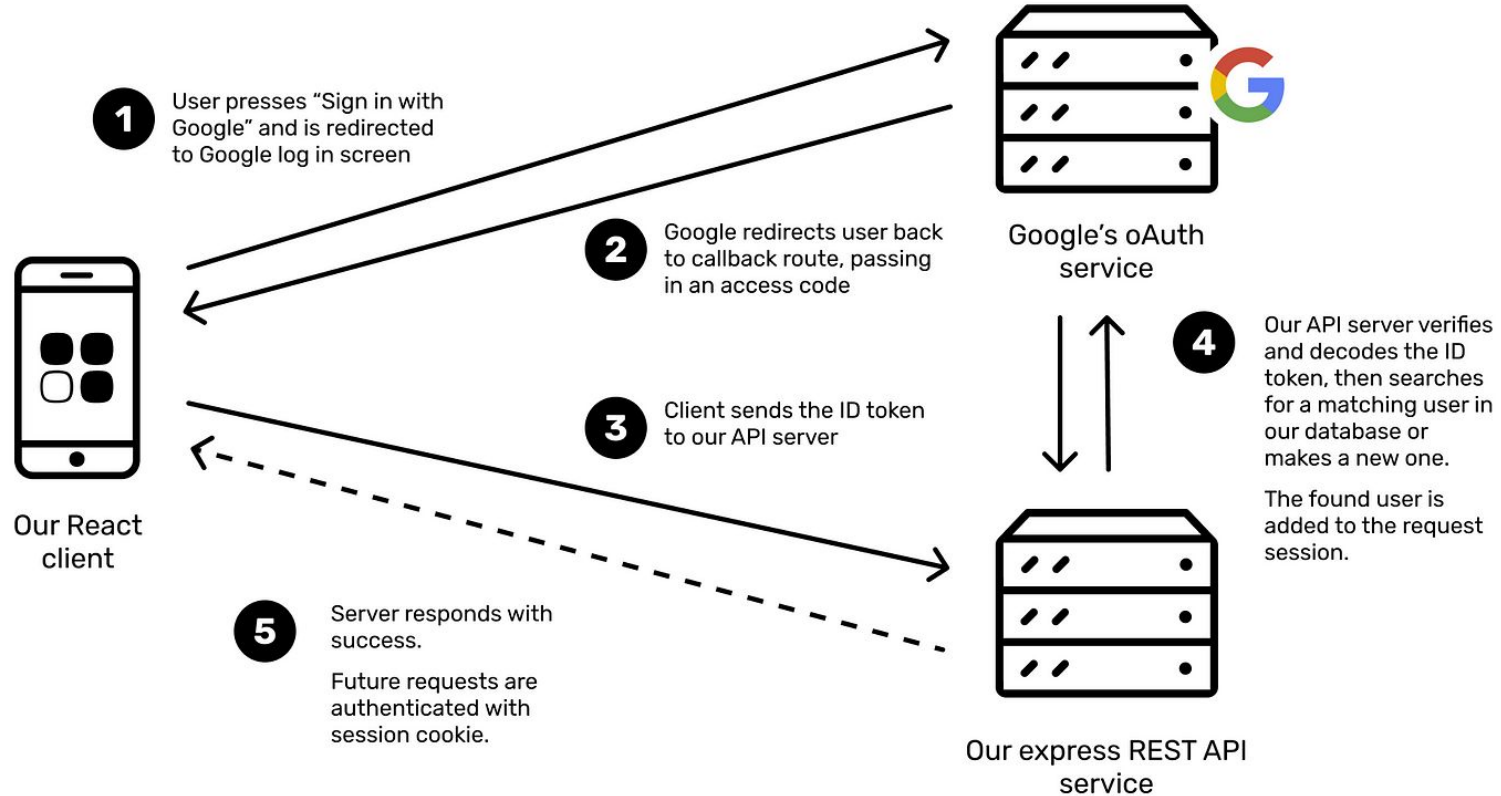
Token Based Authentication





Agenda

- 1 API
Arquitectura, Usos y ejemplos
- 2 Login + Hash
Cifrado de contraseña para almacenamiento seguro
- 3 JWT
JSON Web Token
- 4 **Google OAuth**
Sistemas de autenticación alternativo
- 5 Ejemplo de código - NEXT.js

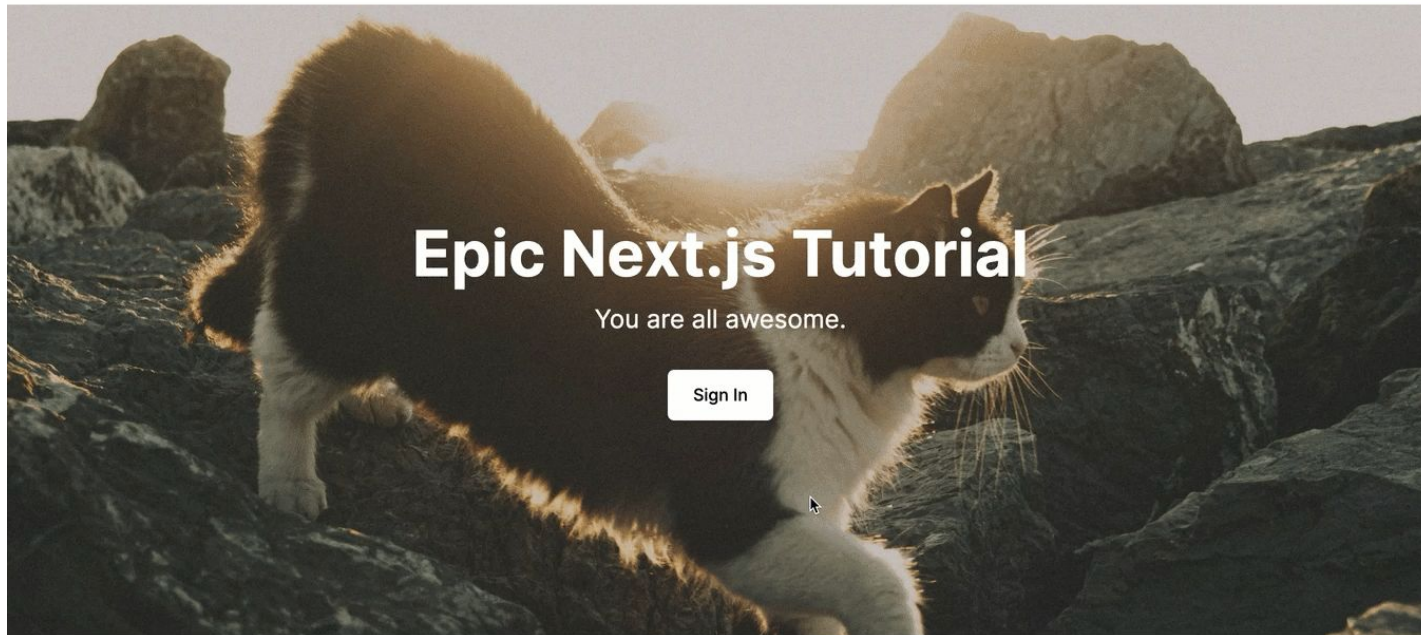


Agenda

- 1 API
Arquitectura, Usos y ejemplos
- 2 Login + Hash
Cifrado de contraseña para almacenamiento seguro
- 3 JWT
JSON Web Token
- 4 Google OAuth
Sistemas de autenticación alternativo
- 5 **Ejemplo de código - NEXT.js**

Summarize AI

Sign In



[/strapi.io/blog/epic-](https://strapi.io/blog/epic-)

